SafeGuard

Sistema de Monitoramento e Detecção de Quedas para Idosos

Felipe Marques Gabriel Ribeiro Pedro Dias

18 de março de 2026

Documento de Projeto Técnico

Sumário

1 Introdução e Contexto **2**

2 Definição do Problema **2**
2.1 Problemas Específicos 2

3 Solução Proposta **2**
3.1 Fluxo da Solução 2

4 Stack Tecnológica **3**

5 Especificação de Hardware **3**
5.1 Microcontrolador 3
5.2 Sensores (Simulação de Comportamento) 3
5.2.1 Lógica de Simulação 3

6 Arquitetura do Sistema **4**
6.1 Publisher (ESP32 – Wokwi) 4
6.2 Broker MQTT (Mosquitto/HiveMQ – Servidor) 4
6.3 Subscriber & Storage (Node-RED + InfluxDB) 4
6.4 Visualization (Grafana) 4

7 Detalhamento da Lógica de Detecção **4**

8 Entregáveis do Projeto **5**

9 Resumo das Tecnologias e Ferramentas **5**

10 Considerações Finais **5**

1 Introdução e Contexto

Com o envelecimento da população, a segurança de idosos em ambientes domésticos ou institucionais torna-se uma preocupação crítica. Quedas são uma das principais causas de lesões e complicações de saúde nessa faixa etária. Muitas vezes, a ausência de um socorro rápido pode agravar as consequências de uma queda.

Este projeto propõe o desenvolvimento de um protótipo IoT (*Internet of Things*) acessível e escalável para detectar quedas em tempo real e notificar cuidadores ou familiares.

2 Definição do Problema

A principal questão que este projeto visa resolver é a **incerteza e a demora no socorro** após um idoso sofrer uma queda, especialmente quando ele está sozinho ou incapaz de acionar ajuda manualmente.

2.1 Problemas Específicos

- **Isolamento:** Idosos que moram sozinhos ficam vulneráveis em situações de emergência.
- **Latência de Comunicação:** A falta de um sistema automático faz com que o tempo entre o incidente e o socorro seja indeterminado, aumentando riscos de saúde.
- **Falta de Dados:** Não há registro histórico da frequência ou intensidade dos movimentos/queda para análise médica preventiva.

3 Solução Proposta

O **SafeGuard** consiste em uma pulseira inteligente (*wearable*) equipada com sensores de movimento e velocidade. O dispositivo monitora os padrões de aceleração do usuário continuamente. Ao detectar um padrão brusco de movimento seguido de ausência de movimento (característico de uma queda), o sistema dispara um alerta automaticamente.

3.1 Fluxo da Solução

1. **Detecção:** O sensor na pulseira identifica variação brusca de velocidade/aceleração (queda).
2. **Processamento:** O microcontrolador analisa os dados e confirma o evento.
3. **Comunicação:** O dado é enviado via protocolo MQTT.
4. **Visualização:** Os dados são armazenados no **InfluxDB** e visualizados em tempo real no **Grafana**.
5. **Ação:** O **Node-RED** recebe o alerta e dispara uma notificação (simulada ou real) para o cuidador.

4 Stack Tecnológica

Para a arquitetura do sistema, utiliza-se um *stack* moderno, leve e amplamente utilizado no mercado de IoT:

- **MQTT** (*Message Queuing Telemetry Transport*): Protocolo de comunicação leve, ideal para dispositivos IoT com largura de banda limitada. Será o barramento de mensagens central.
- **InfluxDB:** Banco de dados de séries temporais (*Time Series Database*). Perfeito para armazenar leituras de sensores contínuos (velocidade, aceleração X, Y, Z) com alta performance.
- **Grafana:** Plataforma de análise e visualização interativa. Permite a criação de *dashboards* para monitorar a saúde do dispositivo e histórico de alertas.
- **Node-RED:** Ferramenta de desenvolvimento baseada em fluxo para conectar dispositivos, APIs e serviços online. Atuará como o "cérebro" lógico para roteamento de alertas.

5 Especificação de Hardware

Como o projeto será desenvolvido em ambiente de simulação, o hardware foi selecionado com base nos componentes disponíveis no **Wokwi**.

5.1 Microcontrolador

ESP32: Escolhido por possuir Wi-Fi e Bluetooth integrados (necessários para o MQTT) e por ser o microcontrolador mais robusto disponível no Wokwi para este fim.

5.2 Sensores (Simulação de Comportamento)

No ambiente Wokwi, utiliza-se a API de código para simular o comportamento físico de um sensor de velocidade/aceleração.

5.2.1 Lógica de Simulação

Como o Wokwi possui limitações de sensores físicos 3D, utilizou-se o código C++ no ESP32 para gerar dados de **Acelerômetro (IMU – Unidade de Medição Inercial)**.

- **Simulação de "Velocidade":** A velocidade será derivada matematicamente a partir da aceleração no código, ou simulam-se picos de aceleração para representar a queda.
- **Interação:** Utiliza-se um **Potenciômetro** ou um **Botão** no Wokwi para simular o "evento de queda" durante os testes.
  - *Exemplo:* Girar o potenciômetro para o máximo simula uma aceleração brusca (impacto), disparando o fluxo de alerta.

6 Arquitetura do Sistema

O fluxo de dados segue a seguinte topologia:

6.1 Publisher (ESP32 – Wokwi)

- Lê o estado simulado do sensor/botão.
- Verifica condição de queda (ex: Aceleração > Limite).
- Publica nos tópicos MQTT:
  - `safeguard/queda/alerta` (Status: ON/OFF)
  - `safeguard/sensores/velocidade` (Dados numéricos)

6.2 Broker MQTT (Mosquitto/HiveMQ – Servidor)

- Recebe as mensagens e as distribui para os inscritos (*Subscribers*).

6.3 Subscriber & Storage (Node-RED + InfluxDB)

- O Node-RED assina o tópico MQTT.
- Os dados numéricos são gravados no InfluxDB.
- Se o tópico de alerta for "ON", o Node-RED envia uma notificação (*Debug node* ou E-mail simulado).

6.4 Visualization (Grafana)

- Conecta ao InfluxDB.
- Exibe gráficos de velocidade ao longo do tempo.
- Exibe um painel de "Status do Sistema" (Normal vs. Queda Detectada).

7 Detalhamento da Lógica de Detecção

Para identificar a queda, o *firmware* no ESP32 implementa uma lógica simplificada de detecção baseada em vetor:

1. **Leitura Contínua:** O sistema lê os valores de aceleração simulados (Eixos X, Y, Z).
2. **Cálculo do Módulo:** Calcula-se a magnitude do vetor aceleração:

$$A = \sqrt{x^2 + y^2 + z^2}$$

3. **Comparação (*Threshold*):**
   - Se $A >$ LIMITE_ALTO (ex: 3g): Indica impacto súbito (possível queda).

- Seguido de $A <$ LIMITE_BAIXO (ex: 0,5g) por alguns segundos: Indica imobilidade pós-impacto.

4. **Ação:** Se as condições acima forem verdadeiras, publica `QUEDA_DETECTADA` no MQTT.

8 Entregáveis do Projeto

1. **Código-fonte (*Firmware*):** Código `.ino` para ESP32 rodando no simulador Wokwi.
2. **Fluxo Node-RED:** Exportação JSON do fluxo de automação.
3. **Dashboard Grafana:** Configuração dos painéis de monitoramento.
4. **Documentação Técnica:** Este documento e manual de uso da simulação.

9 Resumo das Tecnologias e Ferramentas

Tabela 1: Componentes do Sistema

Componente	Tecnologia/Ferramenta	Função no Projeto
Hardware	ESP32 (Wokwi)	Coleta de dados e envio via Wi-Fi
Sensor	Simulação Código + Potenciômetro	Simular aceleração/velocidade e evento
Protocolo	MQTT	Transporte leve das mensagens entre ha
Backend/Logic	Node-RED	Processamento de regras e disparo de al
Banco de Dados	InfluxDB	Armazenamento histórico das leituras (
Frontend	Grafana	Visualização gráfica dos dados e status

10 Considerações Finais

Este documento serve como guia inicial para o desenvolvimento do projeto SafeGuard. A próxima etapa consiste em configurar o ambiente Wokwi e subir as instâncias do Broker MQTT, Node-RED, InfluxDB e Grafana – podendo estas serem instaladas localmente via Docker ou em nuvem.

O sistema proposto oferece uma solução acessível e escalável para um problema crítico de saúde pública, utilizando tecnologias consolidadas no mercado de IoT e permitindo futuras expansões como integração com serviços de emergência e análise preditiva de quedas.